EXPERIMENT - 2

Aim : Create a Blockchain using Python

Theory :

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.

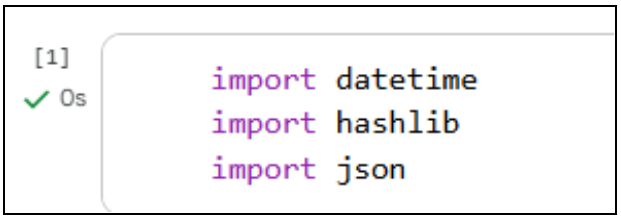
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code :

1. Importing Required Libraries

- datetime → to timestamp each block
- hashlib → to generate SHA-256 hashes
- json → to encode blocks as JSON for hashing
- Flask → to run the blockchain API server



2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
- store the chain
- implement Proof of Work
- validate chain integrity

3. Flask Web API

Provides endpoints:

- /mine_block → mine a new block
- /get_chain → fetch full blockchain
- /is_valid → verify blockchain

Part 1 - Building the Blockchain

```
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash}
        self.chain.append(block)
        return block

    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
```

```
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True
```

Creating a Blockchain

```
blockchain = Blockchain()
```

Mining a new block

```
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
```

```
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash']}
    return response
```

Getting the full Blockchain

```
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return response
```

Checking if the Blockchain is valid

```
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain
is not valid.'}
    return response

print("Functions Menu:")
print("===============")
print("1. Mine a block")
print("2. Display the chain")
print("3. Check the validity of the chain")
choice = int(input("Enter your choice :"))
if choice == 1:
  print(mine_block())
elif choice == 2:
  print(get_chain())
elif choice == 3:
  print(is_valid())
else :
  print("Invalid choice")
```

Output :

```
Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :1
{'message': 'Congratulations, you just mined a block!', 'index': 2,
'timestamp': '2026-08-26 09:03:49.583243', 'proof': 533, 'previous_hash':
'11f4daf563cbd51dbd0d37a8da1a537d7e47ff49bbb08f72571e38f42e0db43b'}

Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :2
{'chain': [{'index': 1, 'timestamp': '2026-08-26 09:01:07.049506',
'proof': 1, 'previous_hash': '0'}, {'index': 2, 'timestamp': '2026-08-26
09:03:49.583243', 'proof': 533, 'previous_hash':
'11f4daf563cbd51dbd0d37a8da1a537d7e47ff49bbb08f72571e38f42e0db43b'}],
'length': 2}

Functions Menu:
===============
1. Mine a block
2. Display the chain
3. Check the validity of the chain
Enter your choice :3
{'message': 'All good. The Blockchain is valid.'}
```

Pretty print ☐


```
{
  "index": 28,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "4389218ffb0ebe26f0ef73422c602630076a5c92ed696a18c8546d0a24233760",
  "proof": 69,
  "timestamp": "2026-08-26 14:49:54.901045"
}
```


Pretty print ☐


```
{
  "index": 30,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "21eda8f78a5a7240d9b708655653a434e0f38855d1fac93364b9de8774332882",
  "proof": 266,
  "timestamp": "2026-08-26 14:50:18.307401"
}
```


Pretty print ☐

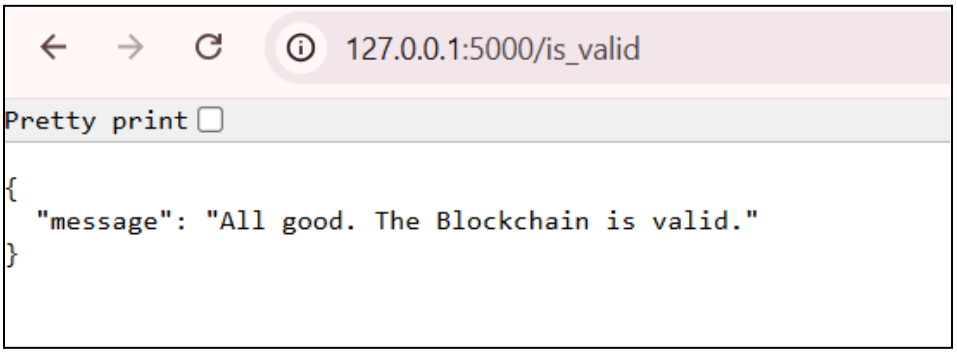

```
{
  "index": 32,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "3d0ea931e9cf5cf3674c2ad7e8ddbe40d798faf30be5f6b443e54831da315f35",
  "proof": 1348,
  "timestamp": "2026-08-26 14:50:32.256076"
}
```


ALL **mine_block**

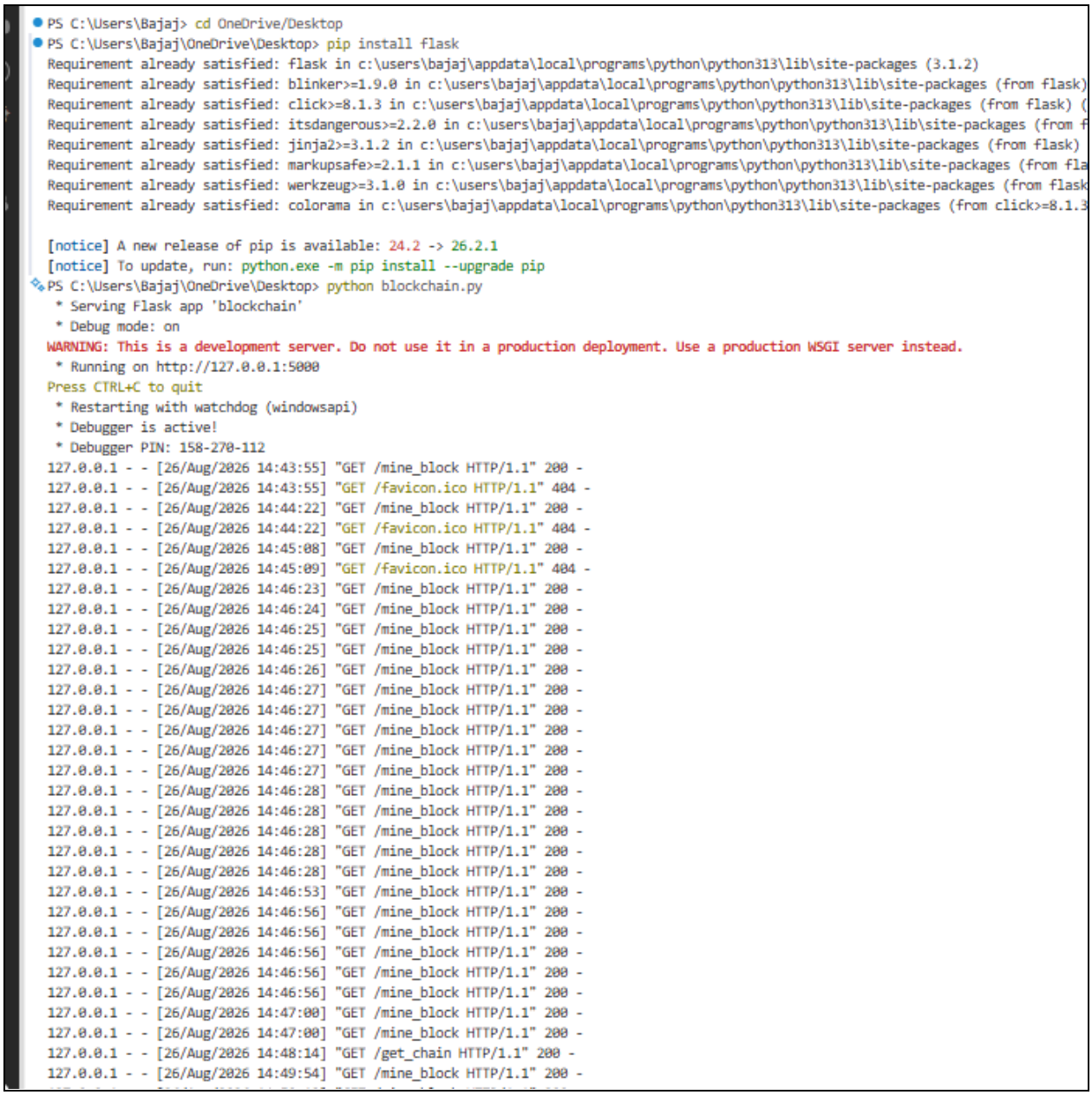
Pretty print

```
    },
    {
      "index": 28,
      "previous_hash": "4389218ffb0ebe26f0ef73422c602630076a5c92ed696a18c8546d0a24233760",
      "proof": 69,
      "timestamp": "2026-08-26 14:49:54.901045"
    },
    {
      "index": 29,
      "previous_hash": "d881b774e1c68e0da9aa7a2daaa4d7c29e7df9e8f7c708f82328084ffbfbd5fe",
      "proof": 103,
      "timestamp": "2026-08-26 14:50:18.122318"
    },
    {
      "index": 30,
      "previous_hash": "21eda8f78a5a7240d9b708655653a434e0f38855d1fac93364b9de8774332882",
      "proof": 266,
      "timestamp": "2026-08-26 14:50:18.307401"
    },
    {
      "index": 31,
      "previous_hash": "de116f36ed84810555cbf4f57793ddac61540d4612b3b31ad242a7b6165ecce2",
      "proof": 10209,
      "timestamp": "2026-08-26 14:50:32.125684"
    },
    {
      "index": 32,
      "previous_hash": "3d0ea931e9cf5cf3674c2ad7e8ddbe40d798faf30be5f6b443e54831da315f35",
      "proof": 1348,
      "timestamp": "2026-08-26 14:50:32.256076"
    },
    {
      "index": 33,
      "previous_hash": "58d5ff54b8809b7294a4a1a4be3d377b10491e2f4802f199d5b2826fb8d13de8",
      "proof": 4188,
      "timestamp": "2026-08-26 14:51:03.242048"
    },
    {
      "index": 34,
      "previous_hash": "70ec66da13cfd23dde47bfb3ed17fc0c0c05f101c80d10e10ee056cd0d322780",
      "proof": 37,
      "timestamp": "2026-08-26 14:51:03.778343"
    }
  ],
  "length": 34
}
```


get_chain



is_valid



Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.